ESTUDO DIRIGIDO · M4 · AULA 2 DE 3 · M4 · EVALS, LOGS E GUARDRAILS NA PRÁTICA

Logs e custo: explicar o que aconteceu e quanto custou

Ao fim deste estudo você explica uma execução do agente a partir do que ela registrou, mede custo e tempo de um lote por sucesso e por percentil e escreve um hook que mede cada ferramenta sem guardar dado de cliente.

Aula de Qua, 25/11/2026 | Tempo sugerido: cerca de 3 horas, em uma ou duas sentadas | Sistemas com IA para Devs · CESAR School

Como usar este estudo

Primeiro leia os conceitos na ordem: cada um sai do anterior, como na aula. Depois faça as práticas, que repetem no seu computador o que o professor demonstrou, com o Claude Code e o caso Balcão. No fim, confira a autoavaliação e as respostas das perguntas.

ANTES DE COMEÇAR

- [ ] Claude Code instalado e com login feito. Teste no terminal: `claude -p "responda apenas: ok"` precisa responder `ok`.
- [ ] Python 3.11 ou mais novo e o SDK do MCP: na pasta `caso-balcao`, rode `pip install -r requirements.txt`.
- [ ] A pasta `caso-balcao` do repositório da turma, github.com/victorhwfreire/sistemas_ia_devs_b2c_20262, no seu computador. Os comandos abaixo partem de `caso-balcao/ferramentas`. No Windows PowerShell, rode antes `[Console]::OutputEncoding = [Text.Encoding]::UTF8` para os acentos saírem certos.
- [ ] Se a pasta `caso-balcao/estado` já tiver arquivos `.jsonl`, apague-os antes da Prática 1 (`Remove-Item ..\estado\*.jsonl`), para os logs terem só as suas execuções.
- [ ] Um editor para abrir arquivos `.jsonl` (o VS Code serve).

O mapa da aula

Pergunta da aula: **quando a resposta sai errada, eu consigo explicar por quê e quanto custou?**



Mapa da aula: cinco conceitos em cadeia. No log útil, cada campo responde a uma pergunta; o trace liga os passos por id; o custo que decide é por sucesso, não por chamada; a latência se lê em percentil e, no agente, soma voltas; a gestão transforma medidas em teto, alerta e dado protegido. Termina na pergunta destacada: quando a resposta sai errada, eu explico o erro e o custo?

Conceito 1 de 5

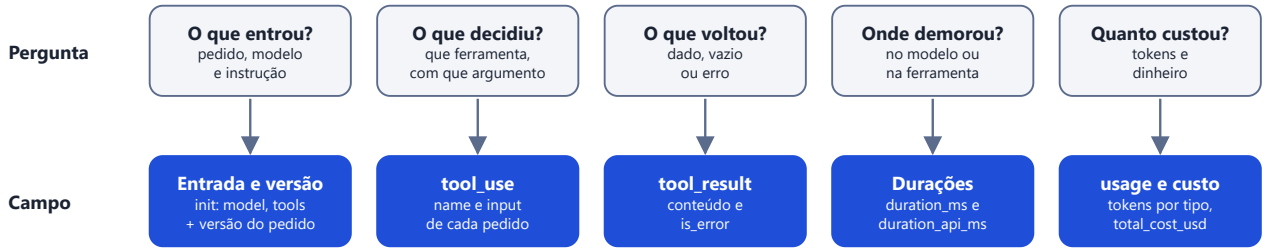
Cada campo do log responde a uma pergunta de incidente

> **Log útil é o registro que, para cada execução, guarda o que entrou, com que versão, o que o modelo decidiu, o que a ferramenta devolveu, quanto tempo levou e quanto custou; cada campo existe para responder a uma pergunta de incidente.**

Um eval, como o da aula 1, diz quantas vezes o sistema acerta. Quando ele erra num caso, a pergunta seguinte é por quê, e o placar não responde. Quem responde é o registro da execução. Para decidir o que registrar, comece pelo incidente: alguém diz que a resposta saiu errada. O que você vai precisar saber?

São cinco perguntas. O que entrou: o pedido, o modelo e a versão da instrução. O que o modelo decidiu: que ferramenta pediu e com que argumentos. O que voltou: o dado, um vazio ou um erro. Onde demorou: no modelo ou na ferramenta. Quanto custou: tokens de cada tipo e dinheiro. Cada campo do log responde a uma delas. Campo que não responde a nenhuma é ruído com custo de armazenamento; pergunta sem campo é incidente que ninguém vai conseguir explicar.

O Claude Code em modo não interativo, com `--output-format stream-json --verbose`, grava boa parte disso: um evento `system` de início (`subtype` `init`) com o modelo e as ferramentas disponíveis; eventos `assistant` com blocos `tool_use` (nome e argumentos); eventos `user` com blocos `tool_result` (o que voltou); e um evento `result` no fim, com `duration_ms`, `duration_api_ms`, `usage` e `total_cost_usd`. O que não vem pronto é a versão da sua instrução: se o pedido muda amanhã, o log de hoje precisa dizer qual versão rodou.



Cinco perguntas de incidente, cada uma com o campo que a responde: o que entrou é respondido pela entrada e pela versão (o evento init, com modelo e ferramentas, mais a versão do pedido); o que o modelo decidiu, pelo tool_use com nome e argumentos; o que voltou, pelo tool_result com conteúdo e is_error; onde demorou, por duration_ms e duration_api_ms; quanto custou, pelo usage com tokens por tipo e pelo total_cost_usd.

NO CASO BALCÃO

No TCK-008, a Loja Bela Casa contesta três tarifas de antecipação. Se o agente não estornar, a pergunta do incidente é "por quê?". A causa pode estar na entrada (o pedido do modo mcp só autoriza estorno de duplicidade), e não no modelo; sem o pedido registrado, ninguém chega a essa conclusão.

CONFUSÃO FREQUENTE

Registrar tudo "por garantia". Log sem pergunta vira volume caro e, como guarda dado de cliente, vira risco. O critério é a pergunta de incidente.

PARA CONFERIR · RESPOSTA NO FIM

Que campo responde à pergunta “o modelo errou ou a ferramenta devolveu dado errado?”

Conceito 2 de 5

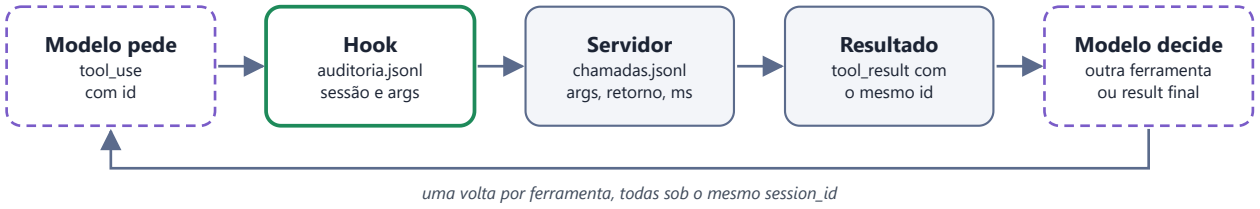
Trace: a árvore de passos ligados por um id

> **Trace é a execução inteira vista como uma árvore de passos (modelo, ferramenta, modelo), em que cada passo se liga ao anterior por um id e registros de lugares diferentes se costuram por chaves em comum.**

Os campos do conceito 1 não vivem soltos. Um pedido de ferramenta só faz sentido ao lado do resultado que voltou e da decisão que veio depois. No agente, a sessão é a raiz; a cada volta, o modelo pede uma ferramenta, a ferramenta executa e o resultado volta para a janela. No stream-json do Claude Code essa ligação é explícita: cada bloco `tool_use` tem um `id`, e o `tool_result` correspondente traz o mesmo valor em `tool_use_id`.

Na prática, os registros nascem em lugares diferentes. No caso Balcão, o Claude Code grava o stream-json; o hook `guarda.py` (ligado com `--limites com`) grava `estado/auditoria.jsonl` antes de cada chamada, com o id da sessão, a ferramenta e os argumentos; e o servidor MCP grava `estado/chamadas.jsonl` depois de executar, com argumentos, retorno e duração em milissegundos, mas sem id de sessão. Montar o trace é costurar: o id da sessão liga o Claude Code ao hook; os argumentos e o horário ligam o hook ao servidor.

Dois detalhes aparecem quando você costura de verdade. O nome da ferramenta muda de um registro para outro: no Claude Code e no hook ela é `mcp__balcao__consultar_cobrancas`; no servidor, só `consultar_cobrancas`. E o hook registra a intenção, antes da execução, enquanto o servidor registra a execução: uma chamada negada pelo hook aparece na auditoria e não aparece no servidor. O OpenTelemetry resolve a costura com um id de trace que viaja entre os componentes; sem ele, você depende de horário e argumentos.



Uma volta do trace em cinco passos: o modelo pede uma ferramenta num tool_use com id; o hook grava a sessão e os argumentos em auditoria.jsonl; o servidor executa e grava argumentos, retorno e ms em chamadas.jsonl; o tool_result volta com o mesmo id; e o modelo decide outra ferramenta ou o result final. A volta se repete, uma por ferramenta, todas sob o mesmo session_id.

NO CASO BALCÃO

No Ato 1, uma chamada a `consultar_cobrancas` para o cliente `CLI-108` aparece em três lugares: como `tool_use` no stream-json, como linha com `decisao` igual a `segue` na auditoria e como linha com o resultado e os `ms` no log do servidor.

CONFUSÃO FREQUENTE

Correlacionar só pelo horário. O log grava segundos, e duas chamadas no mesmo segundo se misturam; argumentos e id da sessão desempatam.

PARA CONFERIR · RESPOSTA NO FIM

Por que uma chamada pode aparecer em `auditoria.jsonl` e não em `chamadas.jsonl`?

Conceito 3 de 5

Custo que decide é por sucesso, não por chamada

> **Custo de uma chamada é a soma dos tokens de entrada, saída, cache criado e cache lido, cada um vezes o seu preço; custo por tarefa soma as chamadas da tarefa; custo por sucesso divide o total pelas tarefas resolvidas.**

Token tem tipo, e cada tipo tem preço. Entrada é o que vai para o modelo; saída é o que ele gera; cache criado é a parte da entrada gravada para reuso; cache lido é a parte reaproveitada de uma chamada anterior. Na documentação da Anthropic, ler do cache custa um décimo do preço de entrada, como regra geral, e gravar no cache de 5 minutos custa 1,25 vez. No agente, cada volta reenvia a janela inteira; é essa repetição que o cache aproveita. E ele não espera a segunda volta: no registro real do agente da aula 1 do M3 (no guia interativo desta aula), a primeira chamada já leu do cache 35.397 dos seus 55.755 tokens de entrada, o começo fixo de instrução e ferramentas, e gravou 20.356 tokens novos.

O evento `result` do stream-json traz `usage` com `input_tokens`, `output_tokens`, `cache_creation_input_tokens` e `cache_read_input_tokens`, e o `total_cost_usd`, que é uma estimativa do Claude Code. Em assinatura, ninguém é cobrado por chamada, mas a estimativa serve para comparar versões e para pôr teto.

Agora as três contas. Custo por chamada é o que uma chamada custou. Custo por tarefa soma todas as chamadas que a tarefa precisou, inclusive as tentativas que falharam. Custo por sucesso divide o custo total pelo número de tarefas resolvidas, sob um critério de sucesso fixo. É a terceira conta que serve para decidir. Exemplo ilustrativo: a versão A gasta R$ 20 em 100 tarefas e resolve 80, R$ 0,25 por sucesso; a versão B gasta R$ 14 e resolve 50, R$ 0,28 por sucesso. B é mais barata por chamada e mais cara por resultado útil.



A conta do custo em cinco etapas: tokens de entrada, saída e cache; cada tipo multiplicado pelo seu preço; o resultado é o custo por chamada, uma volta do modelo; o custo por tarefa soma as voltas e as tentativas; e o custo por sucesso divide o total pelas tarefas resolvidas.

NO CASO BALCÃO

O `workflow_triagem.py` imprime no fim o custo total e o custo por acerto, que é o total dividido pelos tickets que conferem com o gabarito. Se 12 de 12 conferem, as duas contas coincidem; com um erro, o custo por acerto sobe, porque o ticket errado também foi pago.

CONFUSÃO FREQUENTE

Comparar o custo de uma execução do agente com o de uma chamada do workflow. São tarefas diferentes (resolver × triar) e cada lado precisa do seu critério de sucesso.

PARA CONFERIR · RESPOSTA NO FIM

Três execuções do agente custam US$ 0,06, US$ 0,08 e US$ 0,10, e só duas resolvem o ticket. Qual o custo por sucesso?

Conceito 4 de 5

Latência se lê em percentil, e o agente soma voltas

> **Latência é o tempo até a tarefa concluída; num lote, ela se descreve por p50 e p95, não por média, e num agente ela é a soma das voltas.**

O evento `result` traz dois tempos: `duration_ms`, o total da execução, e `duration_api_ms`, o tempo esperando a API do modelo. A diferença é todo o resto: subir o processo, subir o servidor MCP, executar as ferramentas, rodar os hooks. No caso Balcão, o servidor mede o próprio tempo em `chamadas.jsonl` (campo `ms`) e lê dados em memória; quando a execução demora, o log mostra se o tempo foi da ferramenta ou da espera pelo modelo. No agente, cada volta é mais uma espera: execução com mais voltas é execução mais lenta.

As voltas não pesam igual. No registro real do agente da aula 1 do M3, foram três chamadas ao modelo em cerca de 28 s, e a última, a que escreveu a resposta de 2.384 tokens, ocupou cerca de 19 s. Antes de cortar voltas, veja qual delas pesa.

Para um lote, a média engana. Uma execução que travou puxa a média para cima e esconde a experiência típica. Use percentis: o p50 é o valor do meio; o p95 é o tempo que só 5% das execuções superam. O `claude_cli.py` do caso calcula o percentil pelo posto mais próximo, então o resultado é sempre um valor medido; com 12 valores, o p95 é o maior deles. Percentil alto com poucas amostras é instável: trate como sinal, não como meta.

Separe também o tempo até o primeiro token do tempo até a tarefa concluída. Com streaming, a interface reage cedo, mas a tarefa pode terminar muito depois. Exemplo hipotético: o painel comemora 0,5 s de primeiro token no p95, enquanto a tarefa só termina em 8 s no p95. Se o usuário precisa da tarefa pronta, o número que importa é o segundo. Em modo `-p`, como no kit, você mede o fim.



O tempo de uma execução do agente em cinco etapas: a partida do processo e do servidor MCP; a volta 1, que espera a API e a ferramenta; a volta 2, igual; a volta n, que espera a API e responde; e a conclusão, em que duration_ms é a soma de tudo.

NO CASO BALCÃO

Na linha final do `workflow_triagem.py --todos --sem-resposta`, o p50 e o p95 são de 12 tickets com uma chamada cada. Abra o `lote.jsonl` e procure o ticket com o maior `ms`: ele é o p95.

CONFUSÃO FREQUENTE

Ler `duration_ms` como tempo do modelo. Ele inclui tudo; o tempo do modelo é `duration_api_ms`.

PARA CONFERIR · RESPOSTA NO FIM

Num lote de 20 execuções, 19 levam 10 s e uma leva 200 s. O que dizem a média, o p50 e o p95?

Conceito 5 de 5

O que não é medido não é gerenciado

> **Gerenciar é transformar cada medida numa regra: teto de custo por tarefa, alerta no p95, custo por sucesso como métrica de produto e registro por campo que não exponha o cliente.**

Medida que não muda nenhuma decisão é só arquivo. As medidas desta aula viram quatro regras. Teto por tarefa: no Claude Code em modo `-p`, `--max-budget-usd` corta a execução que passar do valor; o número sai do custo das execuções boas, com folga. Alerta: no p95 de tempo e de custo, por tipo de ticket, e não na média. Métrica de produto: custo por sucesso ao lado da taxa de acerto do eval.

A quarta regra é sobre o próprio log. O registro que explica o erro guarda o nome do comércio, valores cobrados e o texto do cliente: log também é dado de cliente. Decida por campo. Metadado (ids, ferramenta, horário, duração, tokens, custo) vai sempre. Conteúdo (pedido, argumentos, retorno) vai quando uma pergunta de incidente exige, com acesso restrito e prazo para apagar. Segredo de autenticação nunca vai.

A telemetria do Claude Code segue essa linha: por padrão, o texto do prompt não é registrado, e é preciso ligar uma variável (`OTEL_LOG_USER_PROMPTS=1`) para incluí-lo. É o mesmo princípio da Prática 3: um hook que mede duração e tamanho de cada ferramenta sem gravar o conteúdo.



As medidas da aula (log, trace, custo e tempo) viram quatro regras: teto por tarefa com --max-budget-usd e folga; alerta no p95 de tempo e custo por tipo de ticket; custo por sucesso como métrica de produto, ao lado do acerto; e registro por campo, com metadado sempre e conteúdo só com motivo. Medida que não vira regra é só arquivo.

NO CASO BALCÃO

Para o agente de cobrança do Balcão: teto de US$ 0,50 por execução (o padrão do `rodar_agente.py`); alerta se o p95 de custo dos tickets de `cobranca_indevida` dobrar de uma semana para a outra; custo por estorno correto no painel; e log com ids, ferramenta, valores e tempos, sem o texto do ticket.

CONFUSÃO FREQUENTE

Pôr o alerta na média. A média sobe devagar e esconde a cauda; quando ela se mexe, o problema já é grande.

PARA CONFERIR · RESPOSTA NO FIM

Em que você se baseia para escolher o valor de `--max-budget-usd` de um agente?

Glossário da aula

TERMO	O QUE É
Log de execução	registro do que entrou, do que o modelo decidiu, do que voltou, do tempo e do custo de uma execução.
Trace	a execução como árvore de passos ligados por ids.
`session_id`	id da sessão do Claude Code; aparece no início do stream-json e na entrada dos hooks.
`tool_use` e `tool_result`	pedido de ferramenta feito pelo modelo (com `id`) e o resultado que volta (com o mesmo valor em `tool_use_id`).
Evento `result`	último evento do stream-json: `subtype`, `num_turns`, `duration_ms`, `duration_api_ms`, `usage` e `total_cost_usd`.
Correlação	ligar registros de lugares diferentes por chaves comuns: id da sessão, argumentos, horário.
Cache de prompt	reuso de uma parte da entrada já processada; lida do cache, custa uma fração do preço de entrada.
Custo por sucesso	custo total dividido pelo número de tarefas resolvidas; no `workflow_triagem.py`, custo por acerto.
p50 e p95	o valor do meio e o valor que só 5% das medidas superam.
`duration_ms` × `duration_api_ms`	tempo total da execução × tempo esperando a API do modelo.
Primeiro token	tempo até a resposta começar a sair; não é o tempo até a tarefa terminar.
Hook PostToolUse	comando que o Claude Code roda depois de cada ferramenta; recebe `tool_name`, `tool_input`, `tool_response`, `tool_use_id` e, quando houver, `duration_ms`.
OpenTelemetry	padrão aberto de telemetria (traces, métricas e logs); o Claude Code exporta métricas e eventos nele.

Prática 1 · Leia o trace de uma execução

55 min

Objetivo. Responder às cinco perguntas de incidente sobre uma execução do agente, citando o registro de onde sai cada resposta, e costurar os três logs do caso.

1. Na pasta `ferramentas`, rode o agente no TCK-008 com as ferramentas MCP do Balcão, contrato forte e o hook de auditoria ligado.

```
python rodar_agente.py --ticket TCK-008 --modo mcp --contrato forte --limites com --
saida p1.jsonl
```

2. No resumo que aparece no fim, responda por escrito: o que entrou? que ferramentas, em que ordem? com que argumentos? onde demorou (tempo total × tempo esperando a API)? quanto custou, e quantos tokens de cada tipo?

3. Liste os eventos do registro, um por linha, para ver o esqueleto do trace.

```
Get-Content p1.jsonl | ForEach-Object { $e = $_ | ConvertFrom-Json; "$($e.type)
$($e.subtype)" }
```

4. Abra `p1.jsonl` no editor (no VS Code: `code p1.jsonl`). Encontre o evento `system` de início (`model`, `tools`, `session_id`); cada bloco `tool_use` (`id`, `name`, `input`) e o `tool_result` com o mesmo `tool_use_id`; e o evento `result` (`num_turns`, `duration_ms`, `duration_api_ms`, `usage`, `total_cost_usd`). Procure também o texto do pedido: se ele não estiver no arquivo, anote essa lacuna.

5. Abra os outros dois registros e costure cada chamada: o `session_id` do início deve ser igual a `sessao` na auditoria; os argumentos do `tool_use` devem ser iguais aos da auditoria e aos do servidor.

```
Get-Content ..\estado\auditoria.jsonl -Encoding UTF8
Get-Content ..\estado\chamadas.jsonl -Encoding UTF8
```

6. Na pasta `caso-balcao`, abra o Claude Code (`claude`) e peça uma conferência da sua leitura, com este texto:

> TEXTO PARA O CLAUDE CODE
>
> Leia ferramentas/p1.jsonl (registro stream-json do Claude Code), estado/auditoria.jsonl (log do hook) e estado/chamadas.jsonl (log do servidor MCP). Monte uma tabela com uma linha por chamada de ferramenta: ordem, ferramenta, argumentos, decisão do hook, se o servidor executou e quantos ms o servidor levou. Diga que chave você usou para ligar cada registro. Depois responda, citando o campo de onde tirou cada resposta: por que o agente estornou ou não estornou as tarifas, quanto a execução custou e onde o tempo foi gasto. Não altere nenhum arquivo.

7. Compare a tabela do Claude Code com as suas respostas do passo 2. Onde vocês discordarem, volte ao arquivo e decida quem está certo.

O QUE OBSERVAR

- O hook grava antes da ferramenta; o servidor, depois. Uma chamada negada só aparece na auditoria.
- O nome da ferramenta tem o prefixo `mcp__balcao__` no Claude Code e no hook, e não tem no servidor.
- Compare o `ms` de cada chamada no servidor com o `duration_ms` da execução: onde o tempo foi gasto?
- A explicação da decisão sobre as tarifas está no registro: no pedido, no retorno de `consultar_cobrancas` ou nos dois.

ESTÁ PRONTO QUANDO

Você respondeu às cinco perguntas citando, em cada resposta, o arquivo e o campo de onde ela saiu.

PARA IR ALÉM

Rode o mesmo ticket com `--contrato fraco` (`--saida p1-fraco.jsonl`) e compare os dois traces: quantas chamadas deram erro e quanto o contrato fraco custou a mais em voltas, tokens e dólares.

Prática 2 · Meça um lote: percentis e custo por sucesso 55 min

Objetivo. Medir p50, p95 e custo por acerto do workflow nos 12 tickets e o custo por sucesso do agente no mesmo tipo de ticket, com critério de sucesso escrito.

1. Rode o workflow de triagem nos 12 tickets, só a classificação (uma chamada ao modelo por ticket), gravando um JSON por ticket.

```
python workflow_triagem.py --todos --sem-resposta --saida lote.jsonl
```

2. Copie a linha final: quantos conferem com o gabarito, p50, p95, custo total e custo por acerto. Calcule ao lado o custo por ticket (custo total ÷ 12).

3. Ache os tickets mais lentos do lote e confira que o tempo do primeiro é o p95.

```
Get-Content lote.jsonl -Encoding UTF8 | ForEach-Object { $_ | ConvertFrom-Json } |
Sort-Object ms -Descending | Select-Object -First 3 ticket, ms, custo, acertou
```

4. Rode o agente três vezes na mesma cobrança duplicada, o TCK-002, com as ferramentas MCP.

```
python rodar_agente.py --ticket TCK-002 --modo mcp --saida ag1.jsonl
python rodar_agente.py --ticket TCK-002 --modo mcp --saida ag2.jsonl
python rodar_agente.py --ticket TCK-002 --modo mcp --saida ag3.jsonl
```

5. Compare as três execuções em uma linha cada.

```
python ler_execucao.py ag1.jsonl --json
python ler_execucao.py ag2.jsonl --json
python ler_execucao.py ag3.jsonl --json
```

6. Decida, para cada execução, se ela resolveu o ticket. Critério: exatamente um `emitir_estorno` de 189.9 em `COB-5002` ou `COB-5003` e uma nota registrada. Os argumentos de cada chamada aparecem no resumo completo.

```
python ler_execucao.py ag1.jsonl
```

7. Monte a tabela. Workflow: custo por ticket, custo por acerto, p50 e p95. Agente: custo de cada execução, custo por sucesso (soma dos três custos ÷ execuções que resolveram) e tempo de cada uma. Escreva em uma frase qual desses números você poria no painel do produto, e por quê.

O QUE OBSERVAR

- Com 12 valores, o p95 pelo posto mais próximo é o maior valor do lote.
- O custo por acerto só se afasta do custo por ticket quando algum ticket erra.
- O workflow tria e o agente resolve: as tarefas são diferentes, e o critério de sucesso de cada lado também.
- Voltas, tempo e custo variam entre as três execuções do agente no mesmo ticket.

ESTÁ PRONTO QUANDO

A tabela dos dois lados está preenchida, com o critério de sucesso escrito, e você tem a frase sobre o painel.

PARA IR ALÉM

Rode o lote com `--modelo haiku` (`--saida lote-haiku.jsonl`) e compare custo total e custo por acerto com o lote anterior. O mais barato por chamada foi mais barato por acerto?

Prática 3 · Escreva um hook que mede cada ferramenta 60 min

Objetivo. Criar, com o Claude Code, um hook PostToolUse que grava duração e tamanho do resultado de cada ferramenta, sem conteúdo do cliente, e ligá-lo pelo `--settings`.

1. A cada execução no modo mcp, o `rodar_agente.py` grava em `estado/config/` a configuração que usou. Veja os dois arquivos: o servidor MCP e as permissões (com o hook de auditoria, se a última execução usou `--limites com`).

```
Get-Content ..\estado\config\mcp.json
Get-Content ..\estado\config\settings.json
```

2. Na pasta `caso-balcao`, abra o Claude Code (`claude`) e peça o hook e a configuração, com este texto:

> TEXTO PARA O CLAUDE CODE
>
> Crie ferramentas/medir.py, um hook PostToolUse do Claude Code. Ele lê da entrada padrão o JSON do evento (campos session_id, tool_use_id, tool_name, tool_response e duration_ms, que pode não vir) e acrescenta uma linha em estado/medicoes.jsonl, com o caminho resolvido a partir do próprio arquivo, contendo: quando (data e hora ISO), sessao, tool_use_id, ferramenta, ms (o duration_ms, ou null se não vier) e bytes_resultado (tamanho em bytes do tool_response serializado em JSON UTF-8). Não grave tool_input nem tool_response no arquivo. Termine sempre com código 0, sem imprimir nada. Depois crie ferramentas/medir-settings.json: copie estado/config/settings.json e acrescente em hooks um PostToolUse com matcher mcp__balcao__.* que roda medir.py com o mesmo Python e caminhos absolutos, no mesmo formato de command usado no rodar_agente.py. Mostre os dois arquivos antes de gravar.

3. Antes de aceitar, leia o que ele propôs e confira: o hook grava só metadados (nada de `tool_input` nem `tool_response`)? Sai sempre com código 0? O matcher é `mcp__balcao__.*`? Os caminhos são absolutos?

4. Teste o hook sozinho, com um evento de mentira, e veja a linha gravada.

```
'{"session_id":"teste","tool_use_id":"toolu_teste","tool_name":"mcp__balcao__buscar_cli
ente","tool_input":{"nome":"Pilates"},"tool_response":{"cliente_id":"CLI-
102"},"duration_ms":12}' | python medir.py
Get-Content ..\estado\medicoes.jsonl
```

5. Rode o agente com a sua configuração. O comando repete as opções que o `rodar_agente.py` usa no modo mcp, trocando o `settings.json` pelo seu. O `>` do Windows PowerShell grava em UTF-16; o `ler_execucao.py` lê os dois formatos.

```
claude -p "Você é o agente de suporte do Balcão. Resolva o ticket abaixo usando as
ferramentas do Balcão. Estorno só para cobrança duplicada comprovada nos lançamentos,
no valor do lançamento. Termine com o que você verificou, o que fez e a resposta ao
cliente. Ticket TCK-002 · Estúdio Pilates Vida: Fui cobrada duas vezes da mensalidade
do plano Pro em outubro. O extrato mostra dois lançamentos de R$ 189,90 no dia 05/10.
Quero o estorno de um deles." --output-format stream-json --verbose --permission-mode
dontAsk --max-budget-usd 0.50 --no-session-persistence --mcp-config
..\estado\config\mcp.json --strict-mcp-config --settings medir-settings.json --
disallowedTools Read Glob Grep Bash Edit Write NotebookEdit WebFetch WebSearch >
p3.jsonl
```

6. Leia o resumo, as medições e as últimas linhas do log do servidor.

```
python ler_execucao.py p3.jsonl
Get-Content ..\estado\medicoes.jsonl -Encoding UTF8
Get-Content ..\estado\chamadas.jsonl -Encoding UTF8 | Select-Object -Last 5
```

7. Ligue cada linha de `medicoes.jsonl` ao trace: o `tool_use_id` da medição é o `id` de um `tool_use` em `p3.jsonl`, e `sessao` é o `session_id` do início. Para cada ferramenta, anote o `ms` visto pelo Claude Code e o `ms` visto pelo servidor, e explique a diferença em uma frase.

O QUE OBSERVAR

- O PostToolUse roda depois da ferramenta: ele mede e avisa, mas não impede a chamada. Sair com código 2 só mostra a mensagem de erro ao modelo.
- O `duration_ms` do Claude Code mede a ferramenta do lado de quem chama; o `ms` do servidor mede só a função.
- O `tool_use_id` costura o hook ao trace sem depender de horário.
- A medição não guarda nenhum dado do cliente: só ids, nomes, tempos e tamanhos.

ESTÁ PRONTO QUANDO

`estado/medicoes.jsonl` tem uma linha por chamada de ferramenta da sua execução, com `sessao` igual ao `session_id` do trace, e você explicou a diferença entre os dois `ms`.

PARA IR ALÉM

Some `bytes_resultado` por ferramenta e descubra qual delas mais engorda a janela de contexto. Proponha um alerta para quando uma ferramenta devolver mais que um tamanho que você escolher.

Autoavaliação

Marque o que você já consegue fazer sem consultar o material. O que ficar em branco indica o conceito para reler.

- [ ] Consigo listar as cinco perguntas de incidente e o campo do registro que responde a cada uma.
- [ ] Consigo achar, num stream-json, o evento de início, cada `tool_use` com o seu `tool_result` e o evento `result`.
- [ ] Consigo costurar o stream-json com o log do hook e do servidor e dizer que chave usei em cada ligação.
- [ ] Consigo calcular custo por tarefa e por sucesso e explicar por que o barato por chamada pode sair caro.
- [ ] Consigo explicar por que uso p50 e p95 em vez de média, e o que muda com poucas amostras.
- [ ] Consigo propor teto, alerta, métrica e regra de registro por campo para um agente.

Respostas das perguntas

1. Que campo responde à pergunta "o modelo errou ou a ferramenta devolveu dado errado?"
O `tool_result` de cada chamada, comparado com o que o modelo escreveu depois. Se o retorno estava certo e a resposta não, o erro é do modelo; se o retorno já veio errado, é da ferramenta ou do dado.

2. Por que uma chamada pode aparecer em `auditoria.jsonl` e não em `chamadas.jsonl`?
Porque o hook PreToolUse roda antes da ferramenta. Se ele negar a chamada, a ferramenta não executa e o servidor não grava nada.

3. Três execuções do agente custam US$ 0,06, US$ 0,08 e US$ 0,10, e só duas resolvem o ticket. Qual o custo por sucesso?
(0,06 + 0,08 + 0,10) ÷ 2 = US$ 0,12. A execução que falhou entra no numerador.

4. Num lote de 20 execuções, 19 levam 10 s e uma leva 200 s. O que dizem a média, o p50 e o p95?
A média é 19,5 s, quase o dobro da experiência típica; o p50 é 10 s. O p95 pelo posto mais próximo é o 19º valor ordenado, também 10 s: a execução de 200 s só aparece no máximo, e é ela que pede investigação.

5. Em que você se baseia para escolher o valor de `--max-budget-usd` de um agente?
No custo das execuções que deram certo, medido num lote, com folga acima do maior valor normal. Teto abaixo disso corta execuções boas; teto muito acima não limita nada.

Para ler depois

- Claude Code, modo não interativo (headless). https://code.claude.com/docs/en/headless — `claude -p` e os formatos de saída `json` e `stream-json`.

- Claude Code, referência de hooks. https://code.claude.com/docs/en/hooks — a entrada do PostToolUse, com `tool_response`, `tool_use_id` e `duration_ms`.
- Claude Code, monitoramento com OpenTelemetry. https://code.claude.com/docs/en/monitoring-usage — métricas e eventos exportados e o que é registrado por padrão.
- Anthropic, prompt caching. https://platform.claude.com/docs/en/build-with-claude/prompt-caching — preço de leitura e escrita de cache e tempo de vida.
- OpenTelemetry, convenções semânticas de GenAI. https://opentelemetry.io/docs/specs/semconv/gen-ai/ — vocabulário padrão para traces de LLM e de agentes.